CSS – Selectores avanzados

Ya vimos la pasada semana Selectores de varios tipos: **Universal**, de **Etiqueta**, de **Clase**, de **Id**, y por último los de **Atributo**.

También vimos como combinarlos y agruparlos:

- Con una coma significa que son Selectores independientes: `h1, h2 {  }`
- Con un espacio significa "descendiente": `header img{  }`
- Con un "**>**" significa "hijo": `header > h1{  }`
- Con un "**+**" significa "adyacente" o "hermano": `label+input{  }`
- Vimos también la pseudo-clase **:not(),** que no sirve como negación: `h1:not(.headblack){  }`

Vamos ahora a avanzar y ver Selectores más avanzados, de **pseudo-classes**, de **pseudo-estados**, y de **pseudo-elementos**.

5. Pseudo-clases para selección de determinados hijos y hermanos

Las pseudo-clases más importantes para la selección de hijos o hermanos son las que se muestran en la siguiente tabla.

Pseudo-clase	Descripción
:first-child	Primer hijo
:last-child	Último hijo
:nth-child(n)	Enésimo elemento hijo
:nth-of-type(n)	Enésimo hermano de su tipo
:only-child	Hijos únicos
:empty	Elementos que no tienen hijos

Ejemplo pseudoclases first-child, last-child y only-child:

Vamos a utilizar las propiedades first-child, last-child y only-child para seleccionar los párrafos que sean el primer y el último hijo de un contenedor o que sea hijo único.

```
p:first-child {color: red;} /*Todos los párrafos que son primer hijo*/

p:last-child {color: blue;} /*Todos los párrafos que son el último hijo*/

p:only-child {color: green;} /*Todos los párrafos que son hijos únicos*/
```

Ejemplo pseudo-clase nth-child:

Veamos cómo funciona la pseudo-clase nth-child mediante un ejemplo en el que aprovechamos la jerarquía de los elementos para referenciar una etiqueta. En el siguiente código tenemos cuatro elementos <p> que son hermanos entre sí e hijos del elemento .

```
<div>
    <p>Primer párrafo</p>
    <p>Segundo párrafo</p>
    <p>Tercer párrafo</p>
    <p>Cuarto párrafo</p>
</div>
```

Utilizando la pseudo-clase nth-child() podemos seleccionar un hijo específico. El número que indiquemos entre paréntesis será la posición del hijo.

En el siguiente código, seleccionaremos todos los párrafos que sean el primer, segundo y cuarto hijo de un contenedor (div, span, etc.).

```
p:nth-child(1){ color: red; }
p:nth-child(2){ color: green; }
p:nth-child(4){ color: blue; }
```

Primer párrafo

Segundo párrafo

Tercer párrafo

Cuarto párrafo

Ejemplo odd (impar) y even (par):

Los valores odd y even son muy útiles para seleccionar elementos pares e impares. El valor **even** representa elementos en posición par y el valor **odd** representa los elementos en posición impar. En el siguiente ejemplo en el que se ha creado una tabla se muestran en verde las filas impares y en rojo las filas pares.

```
tr:nth-child(odd){ color: green;} /* Filas impares */
tr:nth-child(even){ color: red;} /* Filas pares */
```

Fruta	Unidades
Manzanas	40
Peras	45
Plátanos	30
Fresas	100
Naranjas	200

***Ampliación**: en estas páginas podéis ver prácticamente todos los pseudo-clases..*
https://developer.mozilla.org/en-US/docs/Web/CSS/Pseudo-classes

6. Pseudo-clases para los estados de un elemento

Podemos utilizar diferentes pseudo-clases para definir las propiedades de ciertos elementos con diferentes estados. Uno de los usos más comunes es en los enlaces <a> aunque se pueden aplicar en otros elementos.

Pseudo-clase	Descripción
:link	No visitado por el usuario
:visited	Visitado por el usuario
:hover	Modifica el estilo cuando un elemento apuntador pasa por encima
:active	Se activa cuando el usuario pulsa el elemento
:focus	Se activa cuando tiene el foco sobre el elemento
:active	Se activa cuando el usuario pulsa el elemento

Haz una prueba, escribe una etiqueta a de html, y aplica las siguientes propiedades sobre un enlace y observa las diferencias entre los distintos valores.

a:hover { text-decoration: overline;}

a:active { background-color: yellow;}

a:link { color: green;}

a:visited { color: brown; }



Hay también pseudo-estados para controles de formulario:

:focus	Se activa cuando tiene control toma el foco
:disabled	Se activa si el contro tiene atributo disabled
:checked	En los controles radio y checkbox, cuando el control está ticado (“checked”) También en los controles Option de un Select, cuando estén “selected”

Haz una prueba, escribe un form en html, y dentro diferentes controles, y prueba este css

input:focus, option:focus{ background-color: yellow;**}**

input:disabled{ background-color: darkgrey;**}**

input[type=”radius”]:checked, input[type=”checkbox”]:checked { background-color: green; **}**

***Ampliación*: input[type="checkbox"]:checked + label** { color: blue; }
// en función del estado del control, actuamnos sobre su hermano, que es label.

7. Pseudo-elementos

A diferencia de las pseudo-clases, los pseudo-elementos no describen un estado sino que permiten añadir estilos a una parte específica del documento. Por ejemplo, el pseudo-elemento :first-line selecciona la primera línea del elemento indicado por el selector.

Pseudo-elemento	Descripción
::first-line	Primera línea de texto de un elemento
::first-letter	Primera letra de la primera línea de texto de un elemento
::before	Añade contenido al principio del documento
::after	Añade contenido al final del documento
::selection	Coge la porción del texto que se está seleccionando por el usuario

Importante**: Los pseudo-elementos se pueden escribir con **::** o con solo **:
*Yo opto por escribirlos con solo **":"** por comodidad.*

Aplica las siguientes propiedades sobre diferentes elementos y observa las diferencias entre los distintos resultados.

h2::**first-letter** { color: blue; }

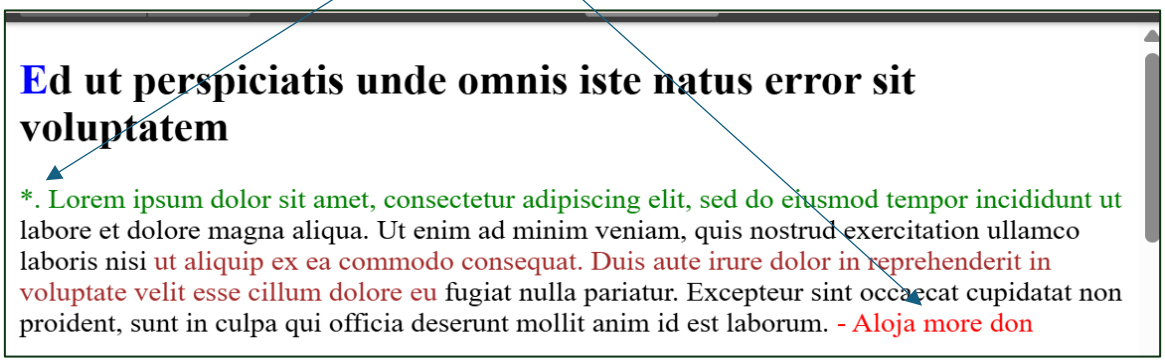
p::**first-line** { color: green; }

p::**selection** { color: brown; }

Vemos también que podemos "inyectar" contenido en el HTML, con las clases ::**after** y ::**before**

p::**after** { **content**: " - Aloja more don"; color: red; }

p::**before** { **content**: "*. "; }



De igual forma, podríamos poner una imagen delante de cada párrafo con ::before y content

```
p::before { content: url("ruta/a/tu/imagen.png"); }
```

Ampliación: en estas páginas podéis ver prácticamente todos los pseudo-elementos...
https://developer.mozilla.org/en-US/docs/Web/CSS/Pseudo-elements

8. Otras Pseudo-clases muy útiles. *Funciones* :not :is :has

`Selector o función de negación :not` *https://developer.mozilla.org/es/docs/Web/CSS/:not*

Sintaxis: `:not(selector) { propiedades de estilo }`

La pseudoclase :not() de CSS representa elementos que no coinciden con el selector simple que se le especifique entre paréntesis.
En teoría también acepta lista de selectores, pero esto no todos los navegadores lo reconocen.

Ejemplos:

Elementos hijos de un article, que no son párrafos

```
article > :not(p) { border: 1px solid blue; }
```

Elementos descendientes de body que no son div ni span (:not se puede concatenar):

```
body :not(div):not(span) { font-weight: bold; }
```

Elementos descendientes de body que no son de la clase .crazy ni .fancy :

```
body :not(.crazy):not(.fancy) { font-family: sans-serif; }
```

Nota: Importante, la sintaxis `body :not(div, span)` *, todavía no está bien soportada*

`La función agregador :is()` *https://developer.mozilla.org/es/docs/Web/CSS/:is*

Sintaxis: `:is(selector, selector, …)`

Imaginemos que queremos que todos los h1, h2 y h3 que sean descendientes de header y de footer para ponerlos de color rojo. Con lo que sabemos, el selector sería muy largo:

```
header h1, header h2, header h3, footer h1, footer h2, footer h3{ color: red; }
```

Hemos tenido que escribir una lista de selectores enorme. Necesitamos una forma compacta. Para esto es la función :is()

La función pseudo-clase **:is()** toma una lista de selectores como argumento y selecciona cualquier elemento que pueda ser seleccionado por uno de los selectores en esa lista. Esto es útil para escribir selectores grandes en una forma más compacta.

Ejemplos:

Selecciona cualquier enlace que esté dentro de lista (ul o ol) o dentro una tabla:

```
:is(ul,ol,table) a { background-color: lime;}
```

Selecciona todos los h1,h2,h3 dentro de un div de clase .amarillo:

```
div.amarillo  :is(h1,h2,h3) { border: 1px solid blue; }
```

Elementos h1 h2 h3, que sean hijos de header o footer

```
:is(header, footer) > :is(h1,h2,h3) { color: red; }
```

La función de cribado :has() https://www.arsys.es/blog/pseudo-clase-css-has

Sintaxis: `selector_principal:has(selector_secundario)`

Nota: Es todavía experimental, pero esperamos que tenga soporte 100% en breve.

Sirve para, una vez hecho un selector (con el selector_principal), describir que otras características debe/n tener el/los objetos seleccionados (mediante el sel secundario).

La pseudo-clase :has() nos sirve para seleccionar elementos que aplican a varios selectores a la vez, siendo alguno de ellos evaluados en el ámbito del selector principal. Por tanto, siempre se selecciona lo de la parte izquierda del :has (selector_principal), siempre que se cumpla lo del paréntesis (selector_secundario).

Ejemplos:

Selecciona párrafos que por lo menos tengan dentro una imagen (como descendiente):

```
p:has(img) {    border: 1px solid #ccc; }
```

Selecciona párrafos que por lo menos tengan una imagen como hija:

```
p:has(> img) {    border: 1px solid #ccc; }
```

Selecciona todos los Formularios que tiene algún campo input inválido:

```
form:has(input:invalid) {  background-color: #ffcccc; }
```

Selector del elemento raíz :root https://developer.mozilla.org/es/docs/Web/CSS/:root

```
:root{ background-color: yellow; } //selecciona el elemento raíz del doc, en este caso <html>
```

*Nota: **:root** en el caso de un doc HTML, será siempre el elemento <html>. Yo lo uso bastante.*

9. Más sobre el selector Universal

El Selector Universal, ***** , es el primero que vimos en este tema.
No obstante, tiene muchos otros usos más allá del típico `* { propiedades }`.

Vemos algunos ejemplos:

```
.card * { margin: 0; } //Selecciona todos los elementos dentro de .card

section *:not(p) { color: gray; } //Selecciona todos los elementos dentro
                                  de <section>, excepto los párrafos <p>

formulario *:focus { outline: 2px solid red; } //Selecciona cualquier elemento
                                                del formulario que tenga el foco

#main-content *::selection { background: yellow; } //texto seleccionado
                                                    dentro de #main-content

.gallery *:hover { transform: scale(1.05); } //cualquier elemento dentro de
                                .gallery al pasar el ratón por encima, le hará zoom.

article > *:nth-child(2){color: red} // cualquier elemento que sea el 2º hijo
                                        de un article
```